Designentscheidungen – UserStory-Tool (Übung 4, Aufgabe 2)

1. Paketstruktur

- Die Klassen sind nach ihrer Aufgabe in Pakete getrennt:
  - `entities`: Fachobjekte (`UserStory`, `Identifiable`)
  - `control`: Verwaltung der Objekte (`Container`, `UserStoryContainer`)
  - `persistence`: Speichern und Laden (Persistenz-Strategien)
  - `view`: Ein- und Ausgabe über die Konsole (`ConsoleView`)
- So bleibt jede Klasse auf ihre Aufgabe beschränkt. Änderungen an einer Schicht (z. B. eine andere Oberfläche) betreffen die anderen nicht.

2. Generischer Container und `UserStoryContainer`

- `Container<T extends Identifiable>` ist generisch, damit er für beliebige Objekte mit ID wiederverwendet werden kann. Würde man ihn für jeden neuen Typ kopieren und anpassen, würden Fehler in jede Kopie mitgeschleppt.
- Über `extends Identifiable` ist sichergestellt, dass jedes Element eine ID hat, die der Container für die Duplikatprüfung braucht.
- Der Singleton steckt in der Unterklasse `UserStoryContainer extends Container<UserStory>`. Ein statisches Attribut kann den Typparameter `T` nicht verwenden, deshalb ist ein Singleton direkt im generischen Container nicht möglich. Die Unterklasse legt den Typ fest und kann so eine einzige typisierte Instanz bereitstellen.

3. Strategy-Pattern für die Persistenz

- Die Persistenz-Strategie wird in `Main` einmal zentral gesetzt, bevor das Programm startet. Der Container wählt sie nicht selbst.
- Der Container kennt dadurch nur das Interface `PersistenceStrategy` und nicht die konkrete Implementierung. Die Strategie (z. B. Datei statt MongoDB) kann ausgetauscht werden, ohne den Container zu ändern.

4. Validierung in `UserStory`

- Die Werte (Mehrwert, Strafe, Aufwand, Risiko) werden in der `UserStory` selbst geprüft, im Konstruktor und in den Settern.
- Dadurch kann die Validierung unabhängig von der Konsole mit JUnit getestet werden, und eine ungültige UserStory kann gar nicht erst entstehen, egal über welche Oberfläche sie angelegt wird.